

Frábært, höldum áfram í næsta skref: "Technical Specification v0.8"

Frábært. Nú förum við úr API-contract yfir í **raunverulega backend implementation specification**. Í v0.8 legg ég áherslu á að skilgreina codebase, domain boundaries, dependency injection, persistence, authorization, events og testing þannig að hægt sé að hefja implementation strax eftir þessa útgáfu.

Technical Specification v0.8

Backend Implementation & Domain Services

Project: Icelandic Legal Case Management System — ILCMS

Version: 0.8

Status: Backend Implementation Baseline

Date: September 2026

1. Purpose

Technical Specification v0.8 skilgreinir hvernig ILCMS backend verður útfært.

v0.7 skilgreindi API contract.

v0.8 skilgreinir:

- Python/FastAPI application structure
- Domain model implementation
- SQLAlchemy persistence
- Pydantic schemas
- Service layer
- Repository layer
- Authorization
- Audit
- Domain events
- Background jobs
- Dependency injection
- Error handling
- Transactions

- Testing
- Configuration
- Initial implementation sequence

Markmiðið er að v0.8 sé nægilega nákvæmt til að developer geti byrjað að byggja kerfið án þess að þurfa að taka grundvallar architectural decisions aftur.

2. Technology Baseline

2.1 Runtime

Python 3.12+

Recommended production baseline:

Python 3.12

Python version skal pinna í CI/CD og container image.

3. Core Framework

FastAPI

SQLAlchemy 2.x

Pydantic 2.x

Alembic

psycpg 3

Supporting libraries:

httpx

pytest

pytest-asyncio

structlog

python-json-logger

Exact dependency versions skulu pinnaðar í lock file.

4. Project Structure

Recommended repository:

```
ilcms/  
├── backend/  
│   └──  
└──
```

```
app/
├── main.py
├── config.py
├── api/
│   ├── router.py
│   ├── dependencies.py
│   └── v1/
│       ├── cases.py
│       ├── parties.py
│       ├── documents.py
│       ├── evidence.py
│       ├── filings.py
│       ├── deadlines.py
│       ├── tasks.py
│       ├── approvals.py
│       ├── search.py
│       ├── audit.py
│       ├── ai.py
│       └── jobs.py
├── domain/
│   ├── cases/
│   ├── documents/
│   ├── evidence/
│   ├── filings/
│   ├── deadlines/
│   ├── approvals/
│   ├── retention/
│   └── audit/
├── services/
│   ├── case_service.py
│   ├── document_service.py
│   ├── evidence_service.py
│   ├── filing_service.py
│   ├── deadline_service.py
│   ├── approval_service.py
│   ├── search_service.py
│   └── audit_service.py
├── repositories/
│   ├── cases.py
│   └── documents.py
```

- evidence.py
- filings.py
- deadlines.py
- users.py

- models/
 - base.py
 - case.py
 - party.py
 - document.py
 - evidence.py
 - filing.py
 - deadline.py
 - approval.py
 - audit.py
 - outbox.py

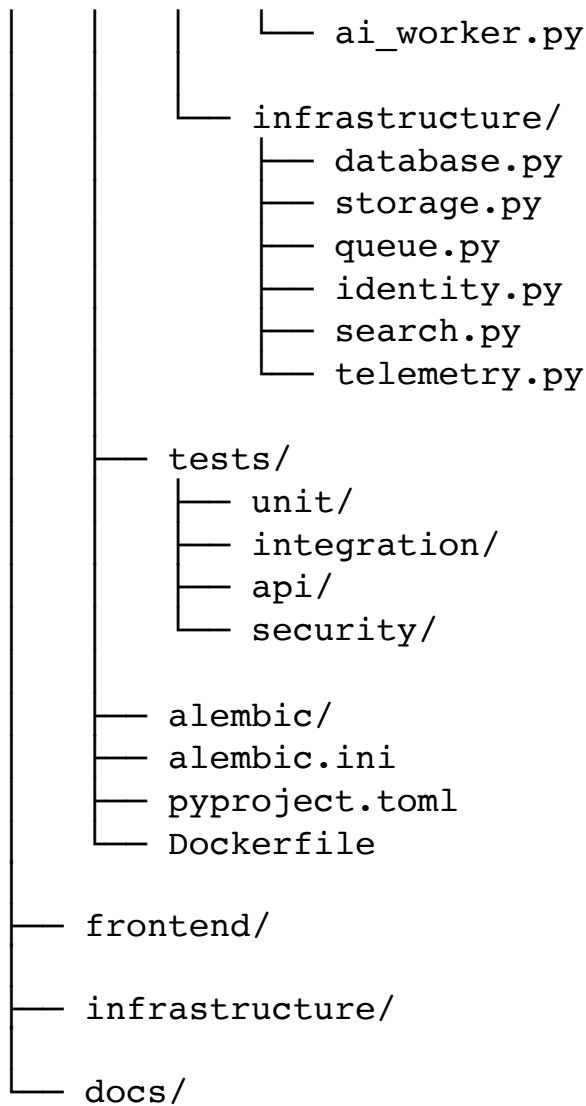
- schemas/
 - cases.py
 - documents.py
 - evidence.py
 - filings.py
 - common.py

- security/
 - authentication.py
 - authorization.py
 - permissions.py
 - principals.py

- workflows/
 - case_workflow.py
 - filing_workflow.py
 - approval_workflow.py

- events/
 - base.py
 - case_events.py
 - document_events.py
 - event_bus.py

- workers/
 - runner.py
 - document_worker.py
 - notification_worker.py



5. Architectural Dependency Rule

Dependencies skulu flæða inn á við.

API

↓

Services

↓

Domain

↓

Repositories / Infrastructure

Domain layer má ekki import-a:

FastAPI

SQLAlchemy

HTTP clients

RabbitMQ

Object storage SDK

þar sem hægt er að komast hjá því.

Þetta heldur domain logic testable og portable.

6. Domain Modules

Primary bounded contexts:

Case Management
Document Management
Evidence Management
Court / Filing
Deadlines
Tasks
Approvals
Retention
Audit
Search
AI
Identity / Access

7. Case Domain

Case er central aggregate.

```
class Case:
    id
    case_number
    title
    description
    case_type
    status
    priority
    confidentiality
    responsible_user_id
```

Case aggregate stjórnar:

- status transitions
- lifecycle rules
- case-level invariants.

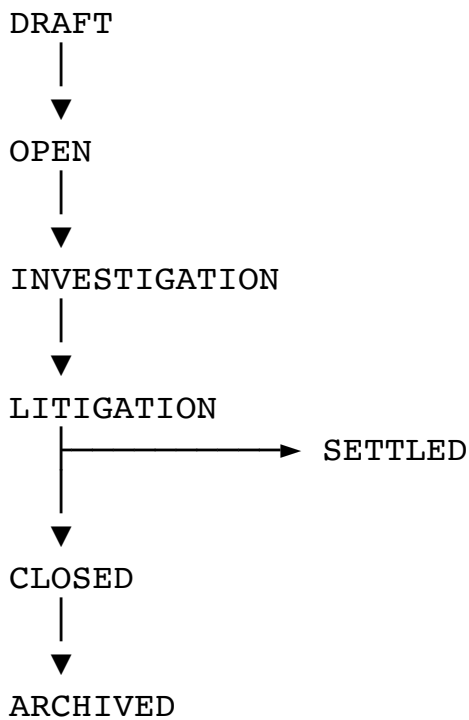
8. Case Status

Enum:

```
class CaseStatus(str, Enum):  
    DRAFT = "DRAFT"  
    OPEN = "OPEN"  
    INVESTIGATION = "INVESTIGATION"  
    LITIGATION = "LITIGATION"  
    SETTLED = "SETTLED"  
    CLOSED = "CLOSED"  
    ARCHIVED = "ARCHIVED"
```

9. Case State Machine

Allowed transitions:



Backend skal reject-a ólöglegar transitions.

Dæmi:

ARCHIVED → OPEN

skal ekki vera mögulegt nema sérstakt administrative recovery workflow sé skilgreint.

10. Domain Method

Ekki:

```
case.status = CaseStatus.CLOSED
```

Heldur:

```
case.close()  
eða:
```

```
case.transition_to(CaseStatus.CLOSED)  
Þetta tryggir að business rules séu alltaf keyrðar.
```

11. Pydantic Request Models

API request:

```
class CreateCaseRequest(BaseModel):  
    case_number: str  
    title: str  
    description: str | None = None  
    case_type: CaseType  
    priority: CasePriority  
    confidentiality: ConfidentialityLevel  
    responsible_user_id: UUID
```

Validation:

```
case_number  
→ required  
→ max length  
→ normalized
```

```
title  
→ required  
→ max length
```

12. Response Models

API response skal vera aðskilið frá SQLAlchemy model.

```
class CaseResponse(BaseModel):  
    id: UUID  
    case_number: str  
    title: str  
    status: CaseStatus  
    priority: CasePriority  
    confidentiality: ConfidentialityLevel  
    created_at: datetime  
    updated_at: datetime
```

Ekki skila ORM object beint út úr API.

13. SQLAlchemy Models

Example:

```
class CaseModel(Base):
    __tablename__ = "cases"

    id: Mapped[UUID] = mapped_column(
        UUID(as_uuid=True),
        primary_key=True
    )

    case_number: Mapped[str] = mapped_column(
        String(100),
        unique=True,
        nullable=False
    )

    title: Mapped[str] = mapped_column(
        String(500),
        nullable=False
    )

    status: Mapped[CaseStatus] = mapped_column(
        Enum(CaseStatus),
        nullable=False
    )
```

14. Database Model vs Domain Model

SQLAlchemy model:

Persistence representation

Domain object:

Business representation

Pydantic model:

API representation

Petta eru þrjú mismunandi responsibilities.

15. Repository Interface

Service skal ekki vita hvernig database er útfærður.

```

class CaseRepository(Protocol):

    async def get(
        self,
        case_id: UUID
    ) -> Case | None:
        ...

    async def create(
        self,
        case: Case
    ) -> Case:
        ...

    async def update(
        self,
        case: Case
    ) -> Case:
        ...

```

16. Repository Implementation

```

CaseRepository
    |
    ▼
PostgresCaseRepository
    |
    ▼
SQLAlchemy
    |
    ▼
PostgreSQL

```

17. Service Layer

CaseService:

```

class CaseService:

    def __init__(
        self,
        cases: CaseRepository,
        authorization: AuthorizationService,
        audit: AuditService
    ):

```

```
):  
    self.cases = cases  
    self.authorization = authorization  
    self.audit = audit
```

18. Create Case Flow

```
HTTP request  
  ↓  
Pydantic validation  
  ↓  
Authentication  
  ↓  
Authorization  
  ↓  
CaseService.create()  
  ↓  
Domain object  
  ↓  
Repository  
  ↓  
PostgreSQL  
  ↓  
Audit event  
  ↓  
Response
```

19. Transaction Boundary

Transaction skal yfirleitt vera á application/service operation level.

Dæmi:

```
BEGIN
```

```
create case  
create audit event  
create outbox event
```

```
COMMIT
```

Ekki:

```
BEGIN  
create case  
COMMIT
```

```
BEGIN
audit
COMMIT
nema explicit reason sé fyrir því.
```

20. Unit of Work

Mælt er með Unit of Work abstraction.

```
class UnitOfWork(Protocol):

    async def commit(self) -> None:
        ...

    async def rollback(self) -> None:
        ...

    async def __aenter__(self):
        ...

    async def __aexit__(self, *args):
        ...
```

Service:

```
async with uow:
    case = await cases.create(case)
    await audit.record(...)
    await uow.commit()
```

21. Dependency Injection

FastAPI dependency injection skal nota til að inject-a:

```
Database session
Current user
Authorization service
Repositories
Services
Dæmi:
```

```
@router.get("/cases/{case_id}")
async def get_case(
    case_id: UUID,
    service: CaseService = Depends(get_case_service)
```

```
):  
    return await service.get(case_id)
```

22. Authentication Principal

Authenticated user verður internal Principal.

```
class Principal:  
    user_id: UUID  
    subject: str  
    roles: set[str]  
    permissions: set[str]
```

Ekki láta domain code vinna beint með JWT claims.

23. Authorization Service

```
class AuthorizationService:  
  
    async def authorize(  
        self,  
        principal: Principal,  
        action: str,  
        resource: object  
    ) -> None:  
        ...
```

Dæmi:

```
await authorization.authorize(  
    principal,  
    "DOCUMENT:DOWNLOAD",  
    document  
)
```

Ef denied:

AuthorizationError

24. Authorization Layers

Authorization skal meta:

1. Authentication
2. Global permission
3. Organization membership
4. Case ACL

5. Confidentiality
6. Legal hold restrictions
7. Resource ownership where applicable
8. Workflow state

25. Security Context

Service operation skal fá security context:

```
SecurityContext(  
    principal=current_user,  
    request_id=request_id,  
    ip_address=ip,  
)  
Petta gerir audit consistent.
```

26. Exception Hierarchy

Application exceptions:

```
ApplicationError  
├── ValidationError  
├── NotFoundError  
├── AuthorizationError  
├── ConflictError  
├── InvalidStateTransitionError  
├── ResourceLockedError  
└── ProcessingError
```

API layer mappar þeim yfir í HTTP responses.

27. No Database Exceptions in API

Ekki:

```
except Exception:  
    return str(exception)
```

Database exceptions skulu:

```
log  
→ classify  
→ map to safe API error
```

28. Audit Service

Allar security-sensitive actions fara í:

```
AuditService.record(...)
```

Dæmi:

```
await audit.record(  
    action="CASE_CREATED",  
    resource_type="case",  
    resource_id=case.id,  
    case_id=case.id,  
    context=context  
)
```

29. Audit Integrity

Audit records skulu vera append-only frá application perspective.

API má ekki hafa:

```
PATCH /audit/{id}  
DELETE /audit/{id}
```

30. Domain Events

Domain event example:

```
@dataclass  
class CaseCreated:  
    event_id: UUID  
    case_id: UUID  
    occurred_at: datetime
```

Events eru business facts.

31. Event Flow

```
CaseService  
    ↓  
CaseCreated  
    ↓  
Outbox  
    ↓  
Message Broker  
    ↓  
Subscribers
```

Possible consumers:

Notification

Search

Analytics

Audit

AI

Integration

32. Outbox Model

Database table:

outbox_events

Fields:

id

event_type

aggregate_type

aggregate_id

payload

created_at

published_at

attempt_count

last_error

33. Outbox Transaction

Example:

BEGIN

INSERT cases

INSERT audit_events

INSERT outbox_events

COMMIT

Petta tryggir atomicity.

34. Outbox Publisher

Worker:

```
SELECT unpublished events
```

↓

```
publish to broker
```

↓

```
mark published
```

```
Ef publish mistekst:
```

```
retry
```

35. Retry Strategy

Exponential backoff:

```
1 sec
```

```
5 sec
```

```
30 sec
```

```
2 min
```

```
10 min
```

```
30 min
```

Maximum attempts skal vera configurable.

Persistent failure fer í dead-letter/error queue.

36. Document Domain

Document aggregate:

Document

- metadata
- versions
- classification
- security
- processing state

Binary content er ekki hluti af PostgreSQL record.

37. Document Storage Interface

```
class ObjectStorage(Protocol):
```

```
    async def put(...):
```

```
        ...
```

```
    async def get(...):
```

```

...

async def delete(...):
    ...

async def create_upload_url(...):
    ...

async def create_download_url(...):
    ...

```

Implementation:

S3ObjectStorage

MinioObjectStorage

38. Document Processing

Document pipeline:

```

UPLOAD
↓
QUARANTINE
↓
MALWARE SCAN
↓
HASH
↓
VALIDATE
↓
OCR
↓
INDEX
↓
AVAILABLE

```

Hvert step skal vera observable.

39. Document Hash

SHA-256:

```
sha256 = hashlib.sha256(content).hexdigest()
```

Hash skal vistað í database.

Purpose:

integrity
duplicate detection
chain-of-custody
export verification

40. Document Versioning

Ný útgáfa:

Document
├── Version 1
├── Version 2
└── Version 3

Version objects eru immutable.

41. Evidence Domain

Evidence tengist:

Case

Document

Party

Source

Evidence metadata skal vera searchable en actual evidence binary skal vera document reference.

42. Court Filing Domain

Filing aggregate:

Filing
├── case
├── filing type
├── documents
├── status
├── submission metadata
└── approval

Submission skal vera stateful workflow.

43. Filing Workflow

DRAFT

↓

REVIEW



APPROVED



SUBMITTED



ACCEPTED

Alternative:

SUBMITTED



REJECTED



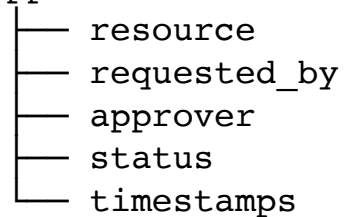
DRAFT

ef court system hafnar filing og workflow leyfir það.

44. Approval Domain

Approval er sérstakt aggregate.

Approval



Important rule:

Sá sem submit-ar critical filing má ekki sjálfkrafa samþykkja eigin filing ef separation-of-duties policy bannar það.

45. Deadline Domain

Deadline:

id

case_id

title

due_at

priority

responsible_user

status

Deadline worker keyrir periodic check:

deadline approaching?

↓

notification

46. Background Worker Architecture

Worker process:

RabbitMQ

↓

Worker

↓

Job handler

↓

Service

Worker má nota sömu application/domain services og API.

47. Worker Isolation

Workers skulu hafa eigin process/container:

ilcms-api

ilcms-worker

ilcms-ocr-worker

ilcms-ai-worker

MVP má sameina suma workers.

48. Job Idempotency

Hvert job hefur:

job_id

event_id

Worker þarf að þola duplicate delivery.

```
if already_processed(event_id):  
    return
```

49. Search Service

Search interface:

```
class SearchService:
```

```

    async def search(
        self,
        query: SearchQuery,
        context: SecurityContext
    ) -> SearchResults:

```

...

Search má aldrei bypass-a authorization.

50. Search MVP

MVP implementation:

PostgreSQL

+

tsvector

+

GIN index

Seinna:

OpenSearch

Search interface verður óbreytt.

51. AI Service

AI abstraction:

```

class AIProvider(Protocol):

```

```

    async def summarize(
        self,
        context: AIContext
    ) -> AIResult:

```

...

Implementations:

LocalLLMProvider

OpenAIProvider

AzureOpenAIProvider

External provider skal vera optional.

52. AI Context Builder

AI fær aldrei raw case object.

Notað:

AIContextBuilder

sem:

authorize

↓

select documents

↓

filter sensitive data

↓

construct context

↓

send to model

53. AI Audit

AI operation:

AI_REQUESTED

AI_COMPLETED

AI_FAILED

AI_RESULT_ACCEPTED

AI_RESULT_REJECTED

Audit record skal tengjast:

user

case

documents used

AI model

timestamp

54. Configuration

Use Pydantic Settings:

```
class Settings(BaseSettings):
    database_url: str
    oidc_issuer: str
    oidc_audience: str
    object_storage_endpoint: str
    queue_url: str
```

Production secrets koma úr:

Vault
Kubernetes Secrets
Cloud secret manager

55. Environment Configuration

Files:

`.env.example`
`.env.test`
EKKI commit-a:

`.env`

56. Database Session

Recommended async SQLAlchemy:

```
engine = create_async_engine(
    settings.database_url,
    pool_pre_ping=True
)
```

Session:

```
async_sessionmaker(
    engine,
    expire_on_commit=False
)
```

57. Database Migrations

Alembic:

```
alembic revision --autogenerate
alembic upgrade head
```

Production:

```
backup
↓
migration
↓
verification
↓
application deployment
```

58. Database Constraints

Business-critical constraints skulu einnig vera í PostgreSQL.

Dæmi:

case_number UNIQUE
document.version UNIQUE per document
required foreign keys
NOT NULL fields
CHECK constraints
Application validation kemur til viðbótar.

59. UUID Strategy

Resources nota UUID.

UUIDv4
eða UUIDv7 ef infrastructure styður það.

UUIDv7 er sérstaklega áhugavert fyrir:

- index locality
- chronological ordering
- distributed generation.

Decision:

Preferred: UUIDv7

Fallback: UUIDv4

60. Timestamps

All timestamps:

UTC

Database:

TIMESTAMPZ

API:

ISO 8601

Example:

2026-09-01T17:00:00Z

UI sér síðan um local timezone presentation.

61. Soft Delete

Legal records skulu ekki almennt vera hard-deleted.

Possible:

deleted_at
deleted_by
deletion_reason

En deletion er háð:

retention
legal hold
permissions
regulatory policy

62. Legal Hold Enforcement

Áður en purge:

RetentionService

↓

LegalHoldService

↓

Is resource protected?

Ef:

YES

þá:

PURGE BLOCKED

63. Logging

Structured JSON logging:

```
{  
  "timestamp": "...",  
  "level": "INFO",  
  "service": "ilcms-api",  
  "request_id": "...",  
  "user_id": "...",  
  "event": "case.created"
```

}

Sensitive data skal vera redacted.

64. Security Logging

Logga:

authentication failures
authorization failures
permission changes
role changes
document download failures
suspicious activity

Ekki logga:

passwords
tokens
document contents
private keys

65. Testing Strategy

Unit tests

Testa:

domain rules
state transitions
authorization logic
validation
services

Integration tests

Testa:

PostgreSQL
repositories
transactions
outbox
object storage
queue

API tests

Testa:

HTTP
authentication
authorization
serialization
status codes

Security tests

Testa sérstaklega:

cross-case access
privilege escalation
search leakage
download bypass
export bypass
workflow bypass
legal-hold bypass

66. Example Domain Unit Test

```
def test_case_cannot_be_archived_from_open():
    case = Case.create(...)

    with pytest.raises(InvalidStateTransitionError):
        case.transition_to(CaseStatus.ARCHIVED)
```

67. Example Authorization Test

```
async def test_user_cannot_access_other_case():
    user = make_user()
    case = make_case(owner_id=other_user.id)

    with pytest.raises(AuthorizationError):
        await service.get(case.id, user)
```

68. API Integration Test

```
POST /api/v1/cases
  ↓
201
  ↓
GET /api/v1/cases/{id}
  ↓
200
```

↓
PATCH
↓
200
↓
GET audit
↓
CASE_CREATED
CASE_UPDATED
Þetta verður fyrsta vertical integration test suite.

69. Test Database

Tests skulu keyra gegn raunverulegu PostgreSQL environment.

Ekki treysta eingöngu á SQLite.

Reason:

PostgreSQL behavior
≠
SQLite behavior

70. Test Containers

Mælt er með:

PostgreSQL container
RabbitMQ container
MinIO container
fyrir integration tests.

71. Code Quality

CI skal keyra:

ruff
mypy
pytest
og:

bandit
fyrir security static analysis.

Optional:

`pre-commit`

72. Type Safety

Backend skal stefna að:

`strict-ish typing`

Public functions og service interfaces skulu vera typed.

Dæmi:

```
async def get_case(
    case_id: UUID,
    context: SecurityContext
) -> CaseResponse:
    ...
```

73. API Documentation

FastAPI OpenAPI verður authoritative API schema.

Development:

`/docs`

`/redoc`

`/openapi.json`

Generated frontend client verður byggður frá OpenAPI.

74. Container

Backend Docker image:

`Python slim base`

↓

`system dependencies`

↓

`Python dependencies`

↓

`application`

Container skal keyra non-root user.

75. Container Security

Image skal:

- keyra sem non-root
- hafa lágmarks packages
- vera scanned
- hafa pinned dependencies
- framleiða SBOM
- ekki innihalda secrets.

76. Graceful Shutdown

API skal:

stop accepting new requests

↓

finish active requests

↓

close DB pool

↓

exit

Workers:

stop consuming

↓

finish current safe job

↓

ack/nack appropriately

↓

exit

77. Health Checks

Kubernetes:

livenessProbe

readinessProbe

startupProbe

FastAPI:

GET /health/live

GET /health/ready

78. Metrics

Initial metrics:

```
http_requests_total
http_request_duration_seconds
http_errors_total
```

```
db_pool_connections
db_query_duration
```

```
queue_messages
queue_failures
```

```
document_processing_duration
document_processing_failures
```

```
ai_requests_total
ai_failures_total
```

79. Trace Context

OpenTelemetry trace:

```
HTTP request
  ↓
CaseService
  ↓
Repository
  ↓
PostgreSQL
og:
```

```
HTTP request
  ↓
Outbox
  ↓
RabbitMQ
  ↓
Worker
```

Request/trace ID verður tenging milli þessara.

80. First Vertical Slice

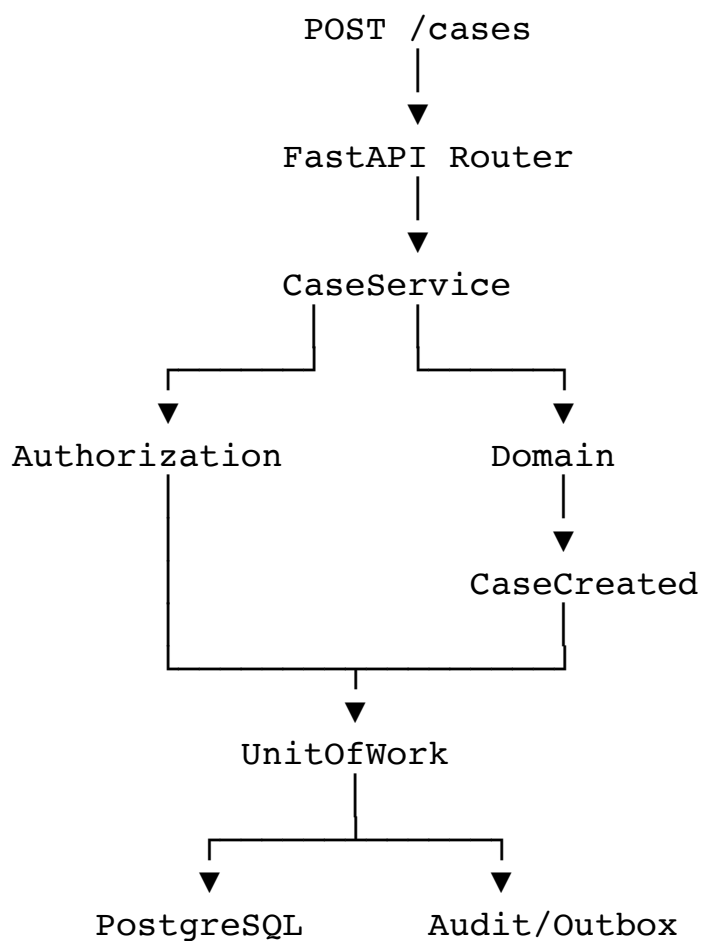
Fyrsta implementation skal ekki reyna að byggja allt kerfið.

Build:

CREATE CASE
GET CASE
LIST CASES
UPDATE CASE
TRANSITION CASE
AUDIT CASE
með:

OIDC/Auth mock
Authorization
PostgreSQL
SQLAlchemy
Alembic
FastAPI
Pytest
OpenAPI

81. First Vertical Slice Architecture



82. Implementation Sequence

Sprint 1

Repository
Python environment
FastAPI
Configuration
Docker
PostgreSQL
Alembic

Sprint 2

Case model
Case repository
Case service
Case API

Sprint 3

Authentication
Authorization
Case ACL

Sprint 4

Audit
Outbox
Events

Sprint 5

Documents
Object storage
Upload workflow

83. MVP Backend Milestones

M1

Application starts
Database connects
Health endpoint works

M2

Case CRUD works

M3

Authentication + authorization works

M4

Audit + outbox works

M5

Documents work

M6

Workflow works

M7

Search works

M8

Production deployment

84. Definition of Done — Backend MVP

Backend MVP telst lokið þegar:

- FastAPI application structure defined
- Domain boundaries defined
- Repository architecture defined
- Service architecture defined
- SQLAlchemy architecture defined
- Pydantic architecture defined
- Dependency injection defined
- Authentication boundary defined
- Authorization service defined
- Security context defined
- Transaction strategy defined
- Unit of Work defined
- Audit service defined

- Domain events defined
- Outbox pattern defined
- Worker architecture defined
- Document storage abstraction defined
- AI provider abstraction defined
- Search abstraction defined
- Configuration strategy defined
- Testing strategy defined
- Container strategy defined
- Observability defined
- First vertical slice defined

85. Architectural Decision — Modular Monolith

Decision: ILCMS backend byrjar sem modular monolith.

One FastAPI application
 +
 Strong domain boundaries
 +
 Independent background workers
 Ekki byrja með microservices.

86. Architectural Decision — Abstractions

External infrastructure skal vera replaceable.

ObjectStorage
 Queue
 Search
 AIProvider
 IdentityProvider
 Application notar interfaces.

Pannig er hægt að skipta:

AWS → MinIO
RabbitMQ → Azure Service Bus
OpenAI → Local LLM
Entra → Keycloak
án þess að endurhanna domain layer.

87. Architectural Decision — PostgreSQL as Source of Truth

PostgreSQL er authoritative fyrir:

Cases
Users
Permissions
Documents metadata
Evidence
Filings
Deadlines
Workflow
Audit metadata
Object storage:

Document binaries
Search:

Derived index
AI:

Derived/generated information

88. Architectural Decision — AI Is Not Authoritative

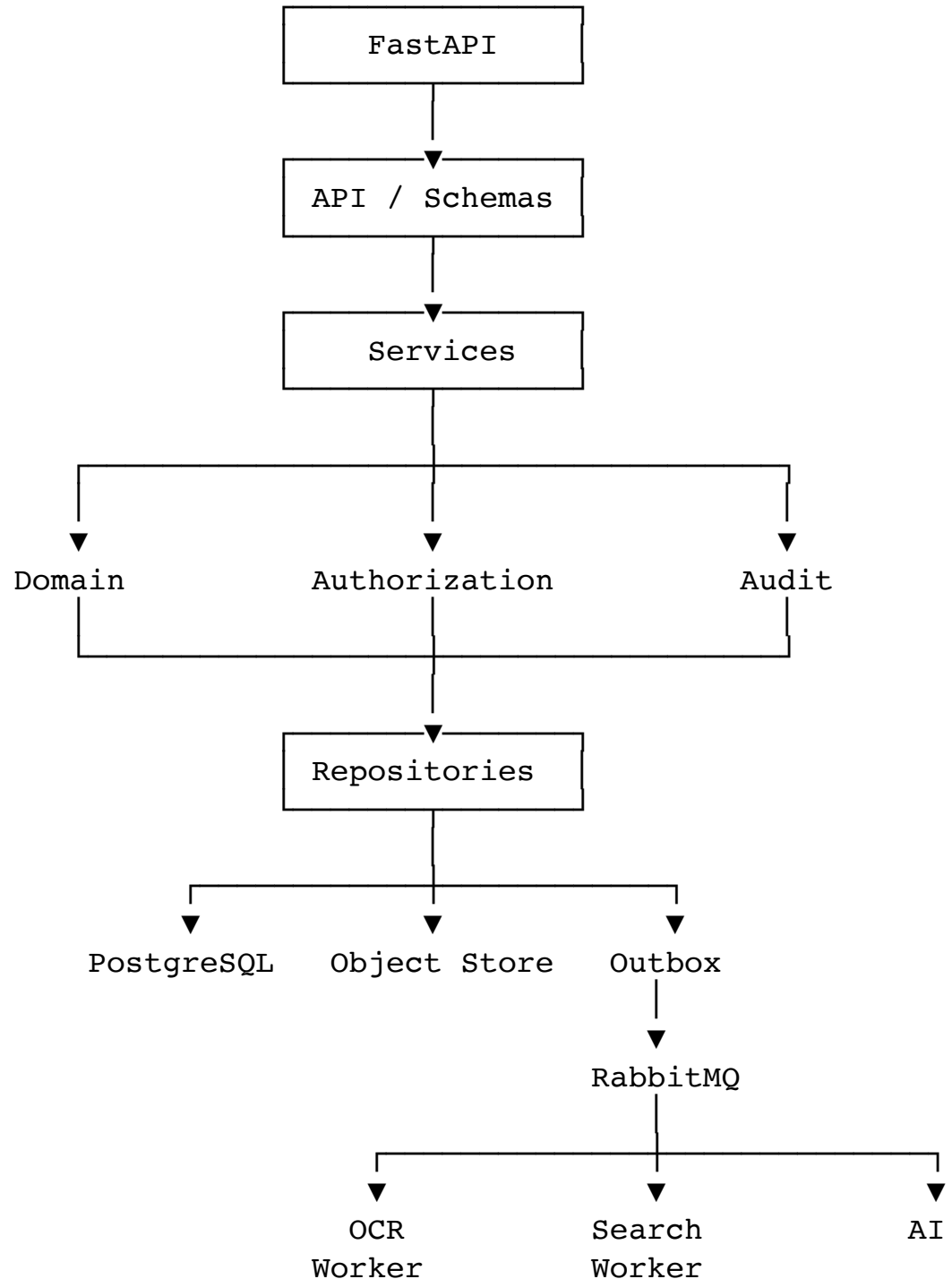
AI output er alltaf:

derived
generated
reviewable
auditable
AI output verður ekki sjálfkrafa:

legal fact
legal conclusion
court filing

án human workflow.

89. Final Backend Architecture



Worker

90. Next Specification — v0.9

Næsta skref ætti að vera:

Technical Specification v0.9

Frontend Architecture & UX Specification

Þar skilgreinum við React application-ið í sömu nákvæmni og backend-ið:

```
frontend/  
├── app/  
├── routes/  
├── components/  
├── features/  
│   ├── cases/  
│   ├── documents/  
│   ├── evidence/  
│   ├── filings/  
│   ├── deadlines/  
│   ├── search/  
│   └── audit/  
├── api/  
├── auth/  
├── permissions/  
├── hooks/  
└── state/
```

Við skilgreinum síðan meðal annars:

- Case Workspace
- Case Dashboard
- Document Browser
- Document Viewer
- Evidence management
- Timeline
- Deadlines
- Filing workflow
- Approval UI
- Search UI
- Audit UI
- Role-aware navigation

- Confidentiality indicators
- Accessibility
- Error handling
- Optimistic/pessimistic UI updates
- React Query data strategy
- TypeScript API client generated frá OpenAPI
- Frontend security boundaries

Síðan, eftir v0.9, getum við tekið **v0.10 — Infrastructure, Docker & Kubernetes Deployment Specification**, sem myndi gera specification-seríuna nánast tilbúna fyrir fyrsta raunverulega development environment.

Við erum nú komin með nokkuð skýra **end-to-end technical blueprint**: v0.1–v0.8 skilgreina gagnalíkan, database, API, workflow/security og application/backend architecture. Næsta mikilvæga skrefið er að hanna **Case Workspace og frontendið** þannig að UI-ið spegli domain modelið í stað þess að verða bara CRUD-form.